PEMROGRAMAN BERORIENTASI OBJEK
CLASS DIAGRAM, INHERITANCE & IMPLEMENTASI PROGRAM



Mohammad Salman Alfarisy **(J0403251118)**
Faris Daffa Al Haq **(J0403251042)**
Syahrul Hidayatullah **(J0403251145)**
Zaky Alnadif **(J0403251057)**
Muhammad Ibnu Akbar **(J0403251028)**

TEKNOLOGI REKAYASA PERANGKAT LUNAK
INSTITUT PERTANIAN BOGOR
2025

1. Analisis Relasi Antar Kelas

Berdasarkan kebutuhan aplikasi perpustakaan, ditemukan tiga bentuk relasi antar kelas, yaitu relasi association antara GenreBuku dengan Buku, relasi aggregation antara Member dengan Buku, serta relasi generalization (inheritance) yang mengelompokkan Member dan Staff sebagai turunan dari Pengguna, dan BukuFisik serta BukuDigital sebagai turunan dari Buku. Berikut adalah rincian analisisnya.

1.1 Relasi GenreBuku dan Buku (Association)

Relasi antara kelas GenreBuku dan Buku merupakan bentuk hubungan Association (Asosiasi Struktural). Penjelasannya adalah sebagai berikut:

- Pola Hubungan: Setiap objek Buku memegang referensi ke satu objek GenreBuku yang mendefinisikan kategori dari buku tersebut (misalnya: Fiksi, Teknologi, atau Sains). Sebaliknya, satu kategori GenreBuku dapat diasosiasikan dengan banyak objek Buku sekaligus.
- Multiplisitas: Buku (0..*) ---memiliki kategori---> (1) GenreBuku.
- Sifat Dependensi: Kedua kelas ini bersifat independen. Objek GenreBuku dapat terdaftar dalam sistem perpustakaan meskipun belum ada objek Buku yang dikaitkan dengannya. Demikian pula, penghapusan data buku tidak serta-merta menghapus data genre dari sistem perpustakaan.

Kelas Sumber / Turunan	Multiplisitas / Relasi	Kelas Tujuan / Induk
Buku	0..* memiliki → 1	GenreBuku

1.2 Relasi Member dan Buku (Aggregation)

Relasi antara kelas Member dan Buku dikategorikan sebagai hubungan Aggregation (Agregasi / Whole-Part yang longgar). Penjelasannya adalah sebagai berikut:

- Pola Hubungan: Kelas Member bertindak sebagai entitas peminjam yang menyimpan koleksi atau daftar dari objek Buku yang sedang dipinjamnya. Objek Buku menjadi bagian dari daftar pinjaman member secara sementara.
- Multiplisitas: Member (1) <>---meminjam--- (0..*) Buku.
- Sifat Dependensi: Relasi ini berbentuk loose coupling. Keberadaan objek Buku tidak bergantung penuh pada objek Member. Jika objek Member dihapus dari sistem, objek

Buku tetap eksis di perpustakaan dan dapat dipinjam oleh member lainnya. Di dalam UML Class Diagram, relasi ini digambarkan dengan simbol wajik/diamond kosong (hollow diamond) pada sisi kelas Member.

Kelas Sumber / Turunan	Multiplisitas / Relasi	Kelas Tujuan / Induk
Member	1 meminjam → 0..*	Buku

1.3 Relasi Generalisasi / Inheritance (Pewarisan)

Selain relasi association dan aggregation, pada pengembangan lebih lanjut aplikasi perpustakaan ditambahkan relasi generalization (inheritance) untuk mengelompokkan kelas-kelas yang memiliki kemiripan atribut dan perilaku ke dalam satu kelas induk (superclass). Tujuannya adalah membuat kode program lebih terstruktur, mudah dikembangkan, dan menghindari duplikasi logika yang sama pada beberapa kelas (prinsip DRY – Don't Repeat Yourself). Terdapat dua struktur generalisasi yang diterapkan pada sistem ini.

1.3.1 Generalisasi Pengguna → Member dan Staff

- Kelas abstrak Pengguna dibuat sebagai superclass yang menyimpan atribut dan perilaku umum yang dimiliki setiap jenis pengguna sistem, yaitu idPengguna dan nama, beserta method abstrak tampilkanInfo() yang wajib diimplementasikan secara berbeda oleh setiap subclass-nya.
- Kelas Member (peminjam buku) dan Staff (pengelola perpustakaan) mewarisi (extends) kelas Pengguna. Keduanya otomatis memperoleh atribut idPengguna dan nama tanpa perlu mendefinisikan ulang, namun masing-masing menambahkan atribut dan perilaku spesifik: Member menambahkan daftarBukuDipinjam beserta method pinjamBuku(), kembalikanBuku(), dan getDaftarBukuDipinjam(); sedangkan Staff menambahkan nomorIdentitas, departemen, gaji beserta method kelolaBuku() dan buatLaporan().
- Sifat Dependensi: karena Pengguna bersifat abstract, kelas ini tidak dapat diinstansiasi secara langsung (pemanggilan new Pengguna(...) tidak valid) — ia hanya berfungsi sebagai kerangka/kontrak bagi subclass-nya. Setiap subclass wajib mengimplementasikan method tampilkanInfo() sesuai konteksnya masing-masing, sehingga sistem dapat memperlakukan Member maupun Staff secara polymorphic melalui referensi bertipe Pengguna.

- Notasi UML: relasi ini digambarkan dengan garis bermata panah segitiga kosong (hollow triangle) yang mengarah dari subclass (Member, Staff) menuju superclass (Pengguna).

Kelas Sumber / Turunan	Multiplisitas / Relasi	Kelas Tujuan / Induk
Member	extends (generalisasi)	Pengguna
Staff	extends (generalisasi)	Pengguna

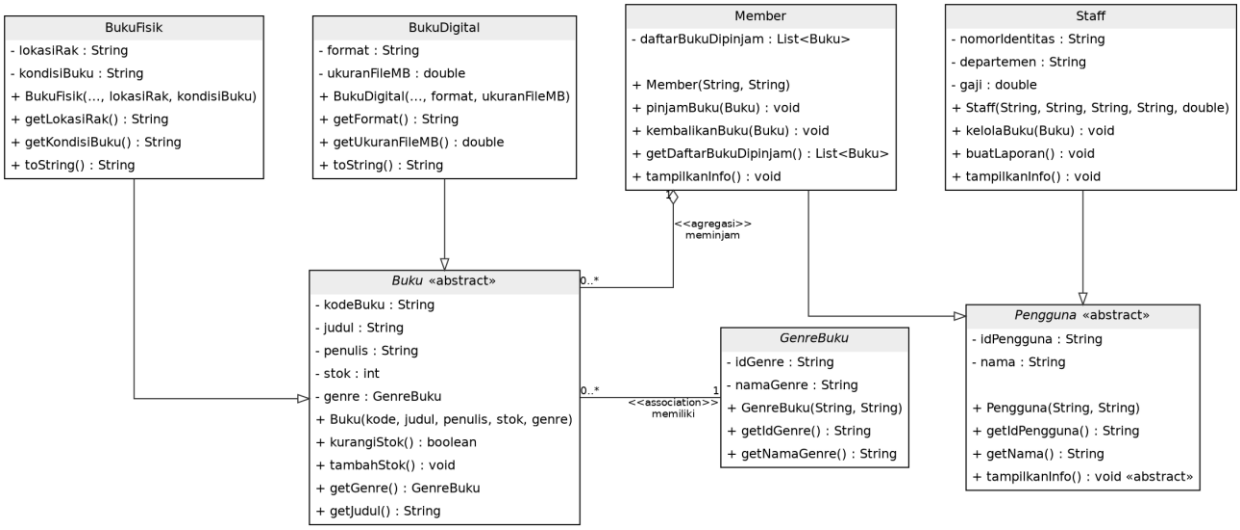
1.3.2 Generalisasi Buku → BukuFisik dan BukuDigital

- Kelas Buku yang semula berupa concrete class kini diubah menjadi kelas abstrak yang menyimpan atribut umum (kodeBuku, judul, penulis, stok, genre) serta perilaku umum seperti kurangiStok() dan tambahStok() yang dipakai bersama oleh seluruh jenis buku.
- Dua subclass diturunkan darinya untuk merepresentasikan dua bentuk koleksi buku di perpustakaan: BukuFisik menambahkan atribut lokasiRak dan kondisiBuku untuk merepresentasikan buku cetak yang tersimpan di rak fisik, sedangkan BukuDigital menambahkan atribut format (misalnya PDF, EPUB) dan ukuranFileMB untuk merepresentasikan buku dalam bentuk berkas digital.
- Kedua subclass meng-override method toString() dengan memanfaatkan super.toString() untuk tetap menampilkan informasi umum dari Buku, kemudian menambahkan informasi spesifik sesuai jenisnya — ini merupakan contoh polymorphism, karena pemanggilan toString() pada objek bertipe Buku akan menghasilkan keluaran berbeda tergantung apakah objek tersebut sebenarnya BukuFisik atau BukuDigital.
- Sifat Dependensi: karena Buku bersifat abstract, seluruh objek buku pada sistem harus diinstansiasi sebagai salah satu subclass-nya (BukuFisik atau BukuDigital), namun tetap dapat diperlakukan secara seragam melalui referensi bertipe Buku (misalnya pada List<Buku> daftarBukuDipinjam di kelas Member) — ini menunjukkan penerapan code reuse sekaligus polymorphism dari inheritance.

Kelas Sumber / Turunan	Multiplisitas / Relasi	Kelas Tujuan / Induk
BukuFisik	extends (generalisasi)	Buku
BukuDigital	extends (generalisasi)	Buku

2. Class Diagram

Gambar 1 berikut merupakan pembaruan dari class diagram sebelumnya dengan penambahan dua struktur generalisasi (inheritance): Pengguna sebagai superclass dari Member dan Staff, serta Buku sebagai superclass dari BukuFisik dan BukuDigital (ditandai dengan notasi garis panah bermata segitiga kosong). Relasi association dan aggregation yang telah dianalisis pada bagian sebelumnya, yaitu GenreBuku–Buku dan Member–Buku, tetap dipertahankan tanpa perubahan.



Gambar 1. Class diagram Aplikasi Perpustakaan setelah penambahan inheritance (Pengguna, Member, Staff, Buku, BukuFisik, BukuDigital, GenreBuku)

3. Implementasi Kode Program

Class diagram pada Gambar 1 diterjemahkan menjadi delapan berkas Java: enam kelas domain yang membentuk dua hierarki inheritance (Pengguna → Member, Staff dan Buku → BukuFisik, BukuDigital), satu kelas independen GenreBuku yang tidak terlibat relasi pewarisan, dan satu kelas Main sebagai program utama untuk mendemonstrasikan seluruh relasi — association, aggregation, maupun generalization/inheritance — sekaligus.

3.1 Kelas Pengguna (Superclass Abstrak)

Pengguna dideklarasikan sebagai abstract class sehingga tidak dapat diinstansiasi secara langsung. Method tampilkanInfo() dideklarasikan sebagai abstract method — hanya berupa signature tanpa isi/implementasi — sehingga setiap subclass diwajibkan menyediakan implementasinya sendiri.

```
public abstract class Pengguna {
    protected String idPengguna;
    protected String nama;
```

```
    public Pengguna(String idPengguna, String nama) {
        this.idPengguna = idPengguna;
        this.nama = nama;
    }

    public String getIdPengguna() {
        return idPengguna;
    }

    public String getNama() {
        return nama;
    }

    public abstract void tampilkanInfo();
}
```

3.2 Kelas Member (extends Pengguna)

Kelas Member mewarisi Pengguna menggunakan keyword extends, kemudian memanggil constructor kelas induk melalui super(idMember, nama) untuk menginisialisasi atribut idPengguna dan nama yang sudah didefinisikan di superclass. Method tampilkanInfo() di-override untuk menampilkan format informasi khusus seorang member, sementara seluruh logika peminjaman/pengembalian buku (relasi aggregation ke Buku) tetap dipertahankan seperti implementasi sebelumnya.

```
import java.util.ArrayList;
import java.util.List;

public class Member extends Pengguna {
    private List<Buku> daftarBukuDipinjam;

    public Member(String idMember, String nama) {
        super(idMember, nama);
        this.daftarBukuDipinjam = new ArrayList<>();
    }

    public List<Buku> getDaftarBukuDipinjam() {
        return daftarBukuDipinjam;
    }

    public void pinjamBuku(Buku buku) {
        String judul = buku.getJudul();
        if (buku.kurangiStok()) {
            daftarBukuDipinjam.add(buku);
            System.out.println(nama + " berhasil meminjam \"" + judul + "\"");
```

```
        } else {
            System.out.println("Maaf, stok \"" + judul + "\" habis.");
        }
    }

    public void kembalikanBuku(Buku buku) {
        if (daftarBukuDipinjam.remove(buku)) {
            buku.tambahStok();
            String judul = buku.getJudul();
            System.out.println(nama + " mengembalikan \"" + judul + "\"");
        }
    }

    public void tampilkanBukuDipinjam() {
        System.out.println("Daftar buku dipinjam oleh " + nama + ":");
        if (daftarBukuDipinjam.isEmpty()) {
            System.out.println(" (tidak ada)");
        } else {
            for (Buku b : daftarBukuDipinjam) {
                System.out.println(" - " + b.getJudul());
            }
        }
    }

    @Override
    public void tampilkanInfo() {
        System.out.println("Member ID: " + idPengguna + ", Nama: " + nama);
    }
}
```

3.3 Kelas Staff (extends Pengguna)

Staff merupakan kelas baru yang juga mewarisi Pengguna. Kelas ini merepresentasikan pengelola perpustakaan dengan atribut tambahan berupa nomorIdentitas, departemen, dan gaji, serta perilaku khas staf seperti kelolaBuku() dan buatLaporan(). Sama seperti Member, Staff memanggil super(idStaff, nama) pada constructor-nya dan meng-override tampilkanInfo() dengan format keluaran yang berbeda.

```
public class Staff extends Pengguna {
    private String nomorIdentitas;
    private String departemen;
    private double gaji;

    public Staff(String idStaff, String nama, String nomorIdentitas, String departemen, double
gaji) {
```

```
        super(idStaff, nama);
        this.nomorIdentitas = nomorIdentitas;
        this.departemen = departemen;
        this.gaji = gaji;
    }

    public String getNomorIdentitas() {
        return nomorIdentitas;
    }

    public void setNomorIdentitas(String nomorIdentitas) {
        this.nomorIdentitas = nomorIdentitas;
    }

    public String getDepartemen() {
        return departemen;
    }

    public void setDepartemen(String departemen) {
        this.departemen = departemen;
    }

    public double getGaji() {
        return gaji;
    }

    public void setGaji(double gaji) {
        this.gaji = gaji;
    }

    public void kelolaBuku(Buku buku) {
        System.out.println(nama + " (Staff) mengelola buku: " + buku.getJudul());
    }

    public void buatLaporan() {
        System.out.println("Laporan dibuat oleh " + nama + " dari departemen " +
departemen);
    }

    @Override
    public void tampilkanInfo() {
        System.out.println("Staff ID: " + idPengguna + ", Nama: " + nama + ", Departemen: "
+ departemen + ", Gaji: Rp" + gaji);
    }
}
```

3.4 Kelas Buku (Superclass Abstrak)

Buku kini dideklarasikan sebagai abstract class. Atribut umum (kodeBuku, judul, penulis, stok, genre) serta method concrete seperti kurangiStok() dan tambahStok() tetap berada di kelas ini dan diwariskan apa adanya oleh kedua subclass-nya, sementara method toString() akan di-override secara berbeda oleh masing-masing subclass.

```
public abstract class Buku {
    protected String kodeBuku;
    protected String judul;
    protected String penulis;
    protected int stok;
    protected GenreBuku genre;

    public Buku(String kodeBuku, String judul, String penulis, int stok, GenreBuku genre) {
        this.kodeBuku = kodeBuku;
        this.judul = judul;
        this.penulis = penulis;
        this.stok = stok;
        this.genre = genre;
    }

    public String getKodeBuku() {
        return kodeBuku;
    }

    public String getJudul() {
        return judul;
    }

    public String getPenulis() {
        return penulis;
    }

    public int getStok() {
        return stok;
    }

    public GenreBuku getGenre() {
        return genre;
    }

    public boolean kurangiStok() {
        if (stok > 0) {
            stok--;
            return true;
```

```
        }
        return false;
    }

    public void tambahStok() {
        stok++;
    }

    @Override
    public String toString() {
        return judul + " (" + genre.getNamaGenre() + ") oleh "
                + penulis + " - stok: " + stok;
    }
}
```

3.5 Kelas BukuFisik (extends Buku)

BukuFisik mewarisi Buku dan menambahkan atribut lokasiRak serta kondisiBuku. Pada constructor-nya, super(kodeBuku, judul, penulis, stok, genre) dipanggil untuk mendelegasikan inisialisasi atribut umum ke kelas induk. Method toString() di-override dengan memanggil super.toString() lalu menambahkan informasi lokasi rak dan kondisi buku fisik.

```
public class BukuFisik extends Buku {
    private String lokasiRak;
    private String kondisiBuku;

    public BukuFisik(String kodeBuku, String judul, String penulis, int stok, GenreBuku
genre, String lokasiRak, String kondisiBuku) {
        super(kodeBuku, judul, penulis, stok, genre);
        this.lokasiRak = lokasiRak;
        this.kondisiBuku = kondisiBuku;
    }

    public String getLokasiRak() {
        return lokasiRak;
    }

    public void setLokasiRak(String lokasiRak) {
        this.lokasiRak = lokasiRak;
    }

    public String getKondisiBuku() {
        return kondisiBuku;
    }

    public void setKondisiBuku(String kondisiBuku) {
```

```
        this.kondisiBuku = kondisiBuku;
    }

    @Override
    public String toString() {
        return super.toString() + " [Fisik - Rak: " + lokasiRak + ", Kondisi: " + kondisiBuku +
"]";
    }
}
```

3.6 Kelas BukuDigital (extends Buku)

BukuDigital juga mewarisi Buku, dengan tambahan atribut format dan ukuranFileMB. Pola implementasinya identik dengan BukuFisik: memanggil super(...) pada constructor dan meng-override toString() untuk menambahkan informasi format serta ukuran berkas digital di belakang informasi umum dari kelas induk.

```
public class BukuDigital extends Buku {
    private String format;
    private double ukuranFileMB;

    public BukuDigital(String kodeBuku, String judul, String penulis, int stok, GenreBuku
genre, String format, double ukuranFileMB) {
        super(kodeBuku, judul, penulis, stok, genre);
        this.format = format;
        this.ukuranFileMB = ukuranFileMB;
    }

    public String getFormat() {
        return format;
    }

    public void setFormat(String format) {
        this.format = format;
    }

    public double getUkuranFileMB() {
        return ukuranFileMB;
    }

    public void setUkuranFileMB(double ukuranFileMB) {
        this.ukuranFileMB = ukuranFileMB;
    }

    @Override
    public String toString() {
```

```
        return super.toString() + " [Digital - Format: " + format + ", Ukuran: " + ukuranFileMB
+ " MB]";
    }
}
```

3.7 Kelas GenreBuku

Kelas GenreBuku tidak mengalami perubahan struktur karena tidak terlibat dalam relasi inheritance — kelas ini tetap berdiri sendiri dan hanya berelasi association dengan Buku seperti pada analisis di bagian 1.1.

```
public class GenreBuku {
private String idGenre;
private String namaGenre;

public GenreBuku(String idGenre, String namaGenre) {
this.idGenre = idGenre;
this.namaGenre = namaGenre;
}
public String getIdGenre() {
return idGenre;
}
public String getNamaGenre() {
return namaGenre;
}
@Override
public String toString() {
return namaGenre;
}
}
```

3.8 Kelas Main

Kelas Main diperbarui untuk mendemonstrasikan hierarki inheritance yang baru: objek buku kini dibuat melalui subclass BukuFisik dan BukuDigital, serta ditambahkan objek Staff di samping dua objek Member. Program menampilkan informasi setiap pengguna (memanfaatkan polymorphism melalui method tampilkanInfo()), menjalankan alur peminjaman dan pengembalian buku, lalu menampilkan status akhir tiap buku serta aktivitas staf dalam mengelola koleksi dan membuat laporan.

```
public class Main {
    public static void main(String[] args) {
        GenreBuku fiksi = new GenreBuku("G01", "Fiksi");
        GenreBuku teknologi = new GenreBuku("G02", "Teknologi");
```

```
        BukuFisik buku1 = new BukuFisik("B01", "Laskar Pelangi",
                "Andrea Hirata", 3, fiksi, "Rak A1", "Baik");
        BukuFisik buku2 = new BukuFisik("B02", "Clean Code",
                "Robert C. Martin", 2, teknologi, "Rak B2", "Baik");
        BukuDigital buku3 = new BukuDigital("B03", "Bumi Manusia",
                "Pramoedya Ananta Toer", 1, fiksi, "PDF", 5.2);

        Member member1 = new Member("M01", "Andi");
        Member member2 = new Member("M02", "Sinta");
        Staff staff1 = new Staff("S01", "Budi", "123456", "Perpustakaan", 5000000);

        System.out.println("=== Informasi Pengguna ===");
        member1.tampilkanInfo();
        member2.tampilkanInfo();
        staff1.tampilkanInfo();
        System.out.println();

        System.out.println("=== Proses Peminjaman ===");
        member1.pinjamBuku(buku1);
        member1.pinjamBuku(buku2);
        member2.pinjamBuku(buku3);
        member2.pinjamBuku(buku3);
        System.out.println();

        member1.tampilkanBukuDipinjam();
        member2.tampilkanBukuDipinjam();
        System.out.println();

        System.out.println("=== Pengembalian Buku ===");
        member1.kembalikanBuku(buku2);
        System.out.println();

        System.out.println("=== Status Buku Setelah Transaksi ===");
        System.out.println(buku1);
        System.out.println(buku2);
        System.out.println(buku3);
        System.out.println();

        System.out.println("=== Aktivitas Staff ===");
        staff1.kelolaBuku(buku1);
        staff1.buatLaporan();
    }
}
```

3.9 Hasil Eksekusi Program

```
=== Informasi Pengguna ===
```

```
Member ID: M01, Nama: Andi
Member ID: M02, Nama: Sinta
Staff ID: S01, Nama: Budi, Departemen: Perpustakaan, Gaji: Rp5000000.0

=== Proses Peminjaman ===
Andi berhasil meminjam "Laskar Pelangi"
Andi berhasil meminjam "Clean Code"
Sinta berhasil meminjam "Bumi Manusia"
Maaf, stok "Bumi Manusia" habis.

Daftar buku dipinjam oleh Andi:
 - Laskar Pelangi
 - Clean Code
Daftar buku dipinjam oleh Sinta:
 - Bumi Manusia

=== Pengembalian Buku ===
Andi mengembalikan "Clean Code"

=== Status Buku Setelah Transaksi ===
Laskar Pelangi (Fiksi) oleh Andrea Hirata - stok: 2 [Fisik - Rak: Rak A1, Kondisi: Baik]
Clean Code (Teknologi) oleh Robert C. Martin - stok: 2 [Fisik - Rak: Rak B2, Kondisi:
Baik]
Bumi Manusia (Fiksi) oleh Pramoedya Ananta Toer - stok: 0 [Digital - Format: PDF,
Ukuran: 5.2 MB]

=== Aktivitas Staff ===
Budi (Staff) mengelola buku: Laskar Pelangi
Laporan dibuat oleh Budi dari departemen Perpustakaan
```

Output di atas membuktikan bahwa program berhasil merepresentasikan seluruh relasi yang dianalisis. Relasi association dan aggregation tetap berfungsi seperti sebelumnya: setiap buku selalu tampil bersama nama genre-nya, dan setiap member dapat memiliki daftar buku pinjaman yang bertambah maupun berkurang tanpa memengaruhi data buku itu sendiri di dalam sistem. Selain itu, output juga membuktikan penerapan generalization/inheritance beserta polymorphism-nya: pemanggilan tampilkanInfo() pada member1, member2, dan staff1 memanggil method dengan nama yang sama namun menghasilkan format keluaran berbeda sesuai override masing-masing subclass dari Pengguna, dan pemanggilan toString() pada buku1/buku2 (BukuFisik) serta buku3 (BukuDigital) menghasilkan format berbeda meski dipanggil melalui referensi yang sama-sama bertipe Buku. Percobaan meminjam “Bumi Manusia” untuk kedua kalinya juga membuktikan validasi stok pada method kurangiStok() — yang diwariskan dari kelas Buku — tetap berjalan dengan benar pada subclass BukuDigital.

4. Tautan Repositori Github

URL Repositori GitHub: https://github.com/Syahruulll/kelompok-12_aplikasi-perpustakaan